```
from google.colab import files
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Upload image
uploaded = files.upload()
file_name = next(iter(uploaded))

# 2. Read the original image
img = cv2.imread(file_name)

# Convert BGR to RGB for display
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# 3. Create ORB feature detector
orb = cv2.ORB_create(nfeatures=500)

# 4. Detect keypoints and descriptors in original image
gray_original = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

kp1, des1 = orb.detectAndCompute(gray_original, None)

# 5. Create Brute-Force Matcher
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

# 6. Define rotation angles
angles = [0, 30, 60, 90, 120]

rotated_images = []
match_images = []
match_counts = []

# 7. Rotate image and perform feature matching
for angle in angles:

    # Get image center
    height, width = img.shape[:2]
    center = (width // 2, height // 2)

    # Create rotation matrix
    rotation_matrix = cv2.getRotationMatrix2D(
        center, angle, 1.0
    )

    # Rotate image
    rotated = cv2.warpAffine(
        img,
        rotation_matrix,
        (width, height)
    )

    # Convert rotated image to grayscale
    gray_rotated = cv2.cvtColor(
        rotated,
        cv2.COLOR_BGR2GRAY
    )

    # Detect ORB features in rotated image
    kp2, des2 = orb.detectAndCompute(
        gray_rotated,
        None
    )

    # Match descriptors
    matches = bf.match(des1, des2)

    # Sort matches according to distance
    matches = sorted(
        matches,
        key=lambda x: x.distance
    )

    # Keep good matches
    good_matches = matches[:50]

    # Draw matching features
    result = cv2.drawMatches(
        img,
        kp1,
        rotated
```

```
        kp2,
        good_matches,
        None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

    # Store results
    rotated_images.append(rotated)
    match_images.append(result)
    match_counts.append(len(good_matches))

# 8. Display rotated images
plt.figure(figsize=(15, 8))

for i in range(len(angles)):

    plt.subplot(2, 3, i + 1)

    plt.imshow(
        cv2.cvtColor(
            rotated_images[i],
            cv2.COLOR_BGR2RGB
        )
    )

    plt.title(
        f"Rotation = {angles[i]}°\n"
        f"Good Matches = {match_counts[i]}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()

# 9. Display feature matching results
plt.figure(figsize=(16, 12))

for i in range(len(angles)):

    plt.subplot(3, 2, i + 1)

    plt.imshow(
        cv2.cvtColor(
            match_images[i],
            cv2.COLOR_BGR2RGB
        )
    )

    plt.title(
        f"Rotation = {angles[i]}° | "
        f"Matches = {match_counts[i]}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()

# 10. Display matching results as a table
print("ROTATION AND FEATURE MATCHING RESULTS")
print("-" * 45)

for angle, count in zip(angles, match_counts):

    print(
        f"Rotation {angle:>3}° : "
        f"{count} good matches"
    )
```